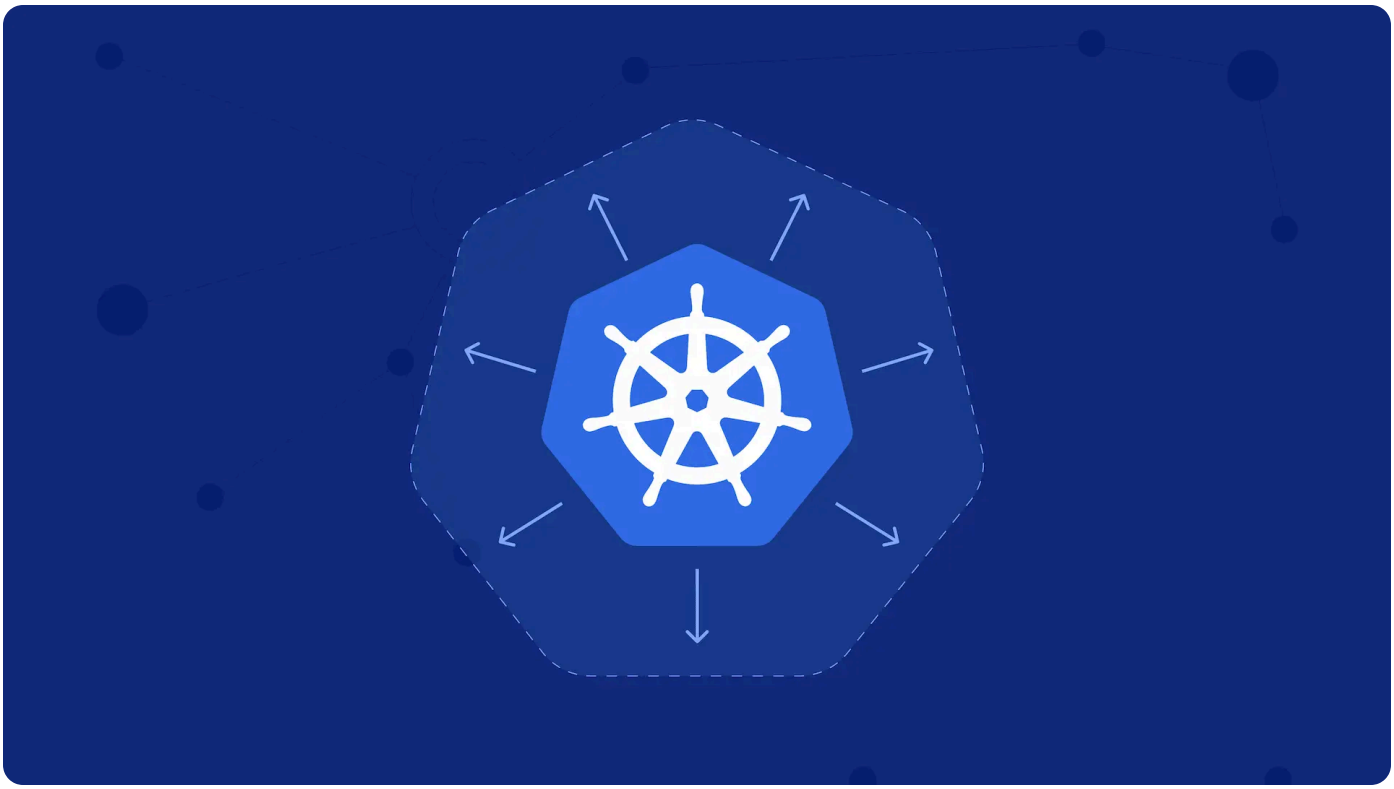
Configuring the Kubernetes Horizontal Pod Autoscaler to scale based on custom metrics from Prometheus

Sreeram Venkitesh
July 22, 2024



Some of the major upsides of using Kubernetes to manage deployments are the self-healing and autoscaling capability of Kubernetes. If a deployment has a sudden spike of traffic, Kubernetes will automatically spin up new containers and

handle that load gracefully. It will also scale down deployments when the traffic reduces.

Kubernetes has a couple of different ways to scale deployments automatically based on the load the application receives. The Horizontal Pod Autoscaler (HPA) can be used out of the box in a Kubernetes cluster to increase or decrease the number of Pods of your deployment. By default HPA supports scaling based on CPU and memory usage, served by the metrics server.

While building NeetoDeploy initially we'd set up to scale deployments based on CPU and memory usage, since these were the default metrics supported by the HPA. However, later we wanted to scale deployments based on the average response time of our application.

This is an example of a case where the metric we want to scale is not directly related to the CPU or the memory usage. Other examples of this could be network metrics from the load balancer, like the number of requests received in the application. In this blog, we will discuss how we achieved autoscaling of deployments in Kubernetes based on the average response time using prometheus-adapter.

When an application receives a lot of requests suddenly, this creates a spike in the average response time. Te CPU and memory metrics also spike but they take longer to catch up. In such cases, being able to scale deployments based on the response time will ensure that the spike in traffic is handled gracefully.

Prometheus is one of the most popular cloud native monitoring tools and the Kubernetes HPA can be extended to scale deployments based on metrics exposed by Prometheus. We used the `prometheus-adapter` to build autoscaling based on average response time in NeetoDeploy.

Setting up the custom metrics

We took following steps to make our HPAs work with Prometheus metrics.

1. Installed `prometheus-adapter` in our cluster.
2. Configured the metric we wanted for our HPAs as a custom metric in the `prometheus-adapter`.
3. Confirmed that the metric is added to the `custom.metrics.k8s.io` API endpoint.
4. Configured an HPA with the custom metric.

Install prometheus-adapter in the cluster

prometheus-adapter is an implementation of the `custom.metrics.k8s.io` API using Prometheus. We used the prometheus-adapter to setup Kubernetes metrics APIs for our Promtheus metrics, which then can be used with our HPAs.

We installed `prometheus-adapter` in our cluster using Helm. We got a template for the values file for the Helm installation here.

We made a few changes to the file before we applied it to our cluster and deployed `prometheus-adapter`:

1. We made sure that the Prometheus deployment is configured properly by giving the correct service url and port.

```
# values.yaml
prometheus:
  # Value is templated
  url: http://prometheus.monitoring.svc.cluster.local
  port: 9090
  path: ""
# ... rest of the file
```

2. We made sure that the custom metrics that we needed for our HPA are configured under `rules.custom` in the `values.yaml` file. In the following example, we are using the custom metric `traefik_service_avg_response_time` since we'll be using that to calculate the average response time for each deployment.

```
# values.yaml
rules:
  default: false

  custom:
    - seriesQuery:'{__name__=~"traefik_service_avg_response_time", service
      resources:
        overrides:
          app_name:
            resource: service
          namespace:
            resource: namespace
      metricsQuery: traefik_service_avg_response_time{<<.LabelMatchers>>}
```

Once we configured our `values.yaml` file properly, we installed `prometheus-adapter` in our cluster with Helm.

```
helm repo add prometheus https://prometheus-community.github.io/helm-chart
helm repo update
helm install prom-adapter prometheus-community/prometheus-adapter --values
```

Query for custom metric

Once we got `prometheus-adapter` running, we queried our cluster to check if the custom metric is coming up in the `custom.metrics.k8s.io` API endpoint.

```
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | jq
```

The response looked like this:

```
{
  "kind": "APIResourceList",
  "apiVersion": "v1",
  "groupVersion": "custom.metrics.k8s.io/v1beta1",
  "resources": [
    {
```

```
      "name": "services/traefik_service_avg_response_time",
      "singularName": "",
      "namespaced": true,
      "kind": "MetricValueList",
      "verbs": ["get"]
    },
    {
      "name": "namespaces/traefik_service_avg_response_time",
      "singularName": "",
      "namespaced": false,
      "kind": "MetricValueList",
      "verbs": ["get"]
    }
  ]
}
```

We also queried the metric API for a particular service we've configured the metric for. Here, we're querying the `traefik_service_avg_response_time` metric for the `neeto-chat-web-staging` app in the default namespace.

```
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1/namespaces/default/s
```

The API response gave the following.

```
{
  "kind": "MetricValueList",
  "apiVersion": "custom.metrics.k8s.io/v1beta1",
  "metadata": {},
  "items": [
    {
      "describedObject": {
        "kind": "Service",
        "namespace": "default",
        "name": "neeto-chat-web-staging",
        "apiVersion": "/v1"
      },
      "metricName": "traefik_service_avg_response_time",
      "timestamp": "2024-02-26T19:31:33Z",
```

```
      "value": "19m",
      "selector": null
    }
  ]
}
```

From the response we can see that the average response time at the instant is reported as `19ms`.

Create the HPA

Now that we're sure that `prometheus-adapter` is able to serve custom metrics under the `custom.metrics.k8s.io` API, we wired this up with a Horizontal Pod Autoscaler, to be able to scale our deployments based on our custom metric.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-name-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app-name-deployment
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Object
      object:
        metric:
          name: traefik_service_avg_response_time
          selector: { matchLabels: { app_name: my-app-name } }
        describedObject:
          apiVersion: v1
          kind: Service
          name: my-app-name
        target:
          type: Value
```

```
            value: 0.03
```

With everything set up, the HPA was able to fetch the custom metric scraped by Prometheus and scale our Pods up and down based on the value of the metric. We also created a recording rule in Prometheus for storing our custom metric queries and dropped the unwanted labels as a best practice. We can use the custom metric stored with the recording rule directly with `prometheus-adapter` to expose the metrics as an API endpoint in Kubernetes. This is helpful when your custom metric queries are complex.

If your application runs on Heroku, you can deploy it on NeetoDeploy without any change. If you want to give NeetoDeploy a try then please send us an email at invite@neeto.com.

If you have questions about NeetoDeploy or want to see the journey, follow NeetoDeploy on X. You can also join our community Slack to chat with us about any Neeto product.

If this blog was helpful, check out our full blog archive.

Stay up to date with our blogs.

Subscribe to receive email notifications for new blog posts.

Enter Your Email Address Subscribe

in mind?

Contact Us

hello@bigbinary.com

SERVICES

Ruby on Rails Development

ReactJS Development

React Native Development

COMPANY

Blog

Case studies

Careers

How we work

Open Source

PRODUCTS

Neeto

BigBinary Academy

©2025 Big Binary LLC. The BigBinary® trademark is owned by Big Binary LLC.